Non-Gaussian Response Distributions

GAM.jl Contributors

Table of contents

Overview

GAMs are not limited to Gaussian responses. By specifying a **family** (distribution) and **link function**, we can model count data, binary outcomes, positive continuous data, and more. The general model is:

$$g(\mu_i) = \beta_0 + f_1(x_{1i}) + \cdots + f_p(x_{pi}), \quad y_i \sim \text{Family}(\mu_i, \phi)$$

This vignette demonstrates three common non-Gaussian families:

Family	Link	Response type	Example
Poisson	log	Counts	Species counts, event rates
Binomial	logit	Binary / proportions	Presence–absence, disease status
Gamma	inverse (or log)	Positive continuous	Waiting times, claim sizes

Setup

```
library(mgcv)
```

```
Loading required package: nlme
```

```
This is mgcv 1.9-3. For overview type 'help("mgcv-package")'.
```

```
library(gratia)
library(ggplot2)
```

Poisson GAM — count data

Simulating count data

We simulate counts from a Poisson distribution with a smooth log-rate:

$$y_i \sim \text{Poisson}(\mu_i), \quad \log(\mu_i) = 1 + 1.5\sin(2\pi x_i)$$

```
set.seed(42)
n <- 300
x <- sort(runif(n))
eta <- 1.0 + 1.5 * sin(2 * pi * x)
mu <- exp(eta)
y_pois <- rpois(n, mu)

df_pois <- data.frame(x = x, y = y_pois)
```

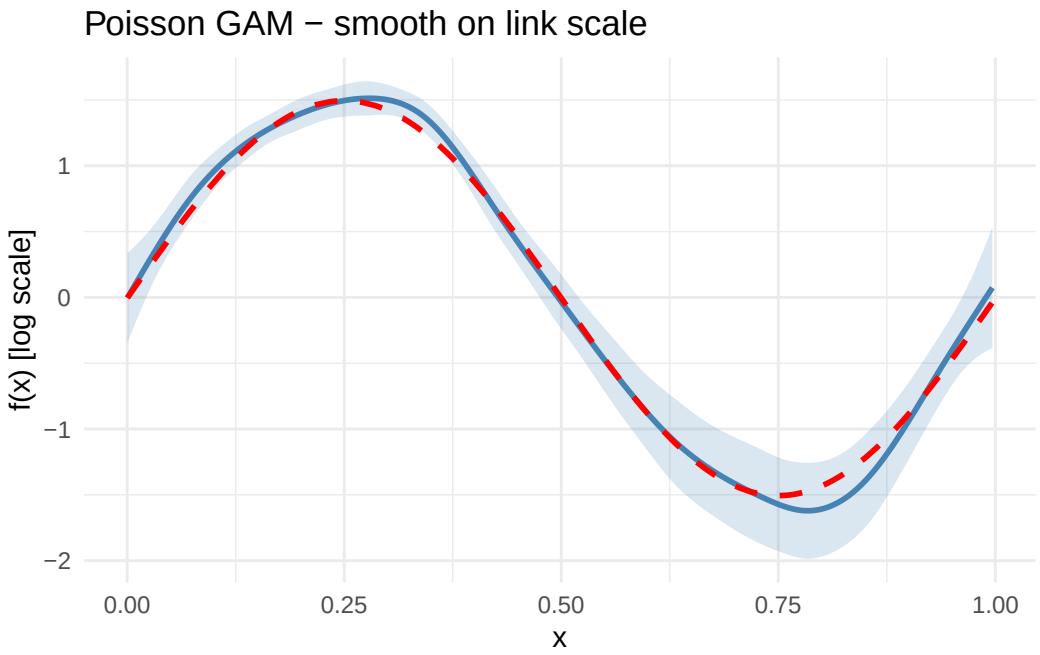
Fitting the model

```
m_pois <- gam(y ~ s(x, k = 15, bs = "cr"), data = df_pois,
              family = poisson(link = "log"), method = "REML")
summary(m_pois)
```

```
Family: poisson
Link function: log

Formula:
y ~ s(x, k = 15, bs = "cr")

Parametric coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  0.93959    0.04685   20.06   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
       edf Ref.df Chi.sq p-value
s(x) 7.995  9.613  719.9  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.764   Deviance explained = 76.4%
-REML = 566.91  Scale est. = 1         n = 300
```

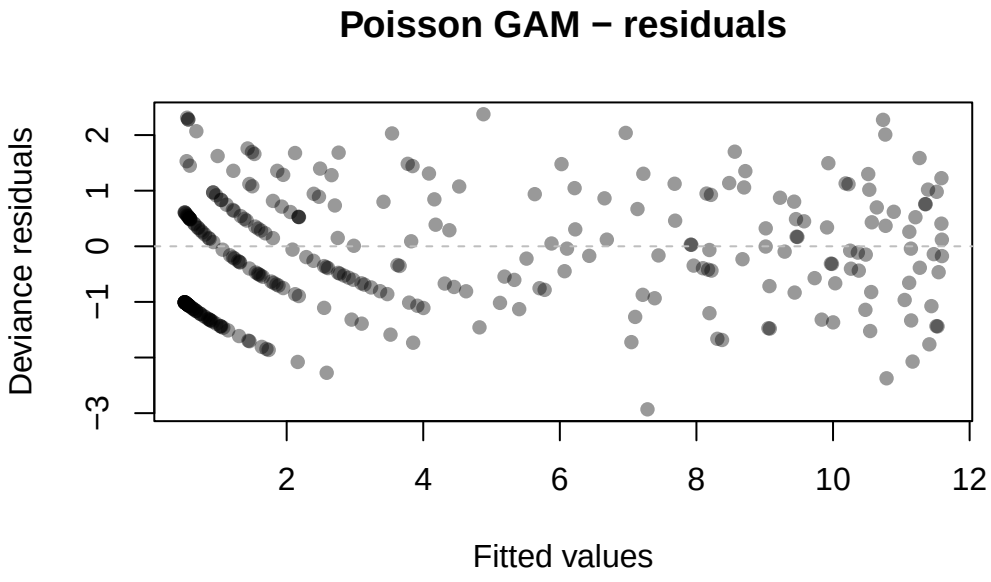
Smooth estimate

```
se_pois <- smooth_estimates(m_pois, n = 200)

ggplot(se_pois, aes(x = x, y = .estimate)) +
  geom_ribbon(aes(ymin = .estimate - 2 * .se, ymax = .estimate + 2 * .se),
              alpha = 0.2, fill = "steelblue") +
  geom_line(linewidth = 1, colour = "steelblue") +
  geom_line(data = data.frame(x = x, y = eta - mean(eta)),
            aes(x = x, y = y), linetype = "dashed", linewidth = 1, colour = "red") +
  labs(x = "x", y = "f(x) [log scale]", title = "Poisson GAM - smooth on link scale") +
  theme_minimal()
```



Deviance residuals

```
resid_pois <- residuals(m_pois, type = "deviance")

plot(fitted(m_pois), resid_pois,
     xlab = "Fitted values", ylab = "Deviance residuals",
     main = "Poisson GAM - residuals", pch = 16, col = adjustcolor("black", 0.4))
abline(h = 0, lty = 2, col = "grey")
```



Binomial GAM — binary data

Simulating binary data

We simulate binary outcomes from a logistic model:

$$y_i \sim \text{Bernoulli}(p_i), \quad \text{logit}(p_i) = -0.5 + 2\sin(2\pi x_i)$$

```
set.seed(42)
x_bin <- sort(runif(n))
eta_bin <- -0.5 + 2.0 * sin(2 * pi * x_bin)
p_true <- plogis(eta_bin)
y_bin <- rbinom(n, 1, p_true)

df_bin <- data.frame(x = x_bin, y = y_bin)
```

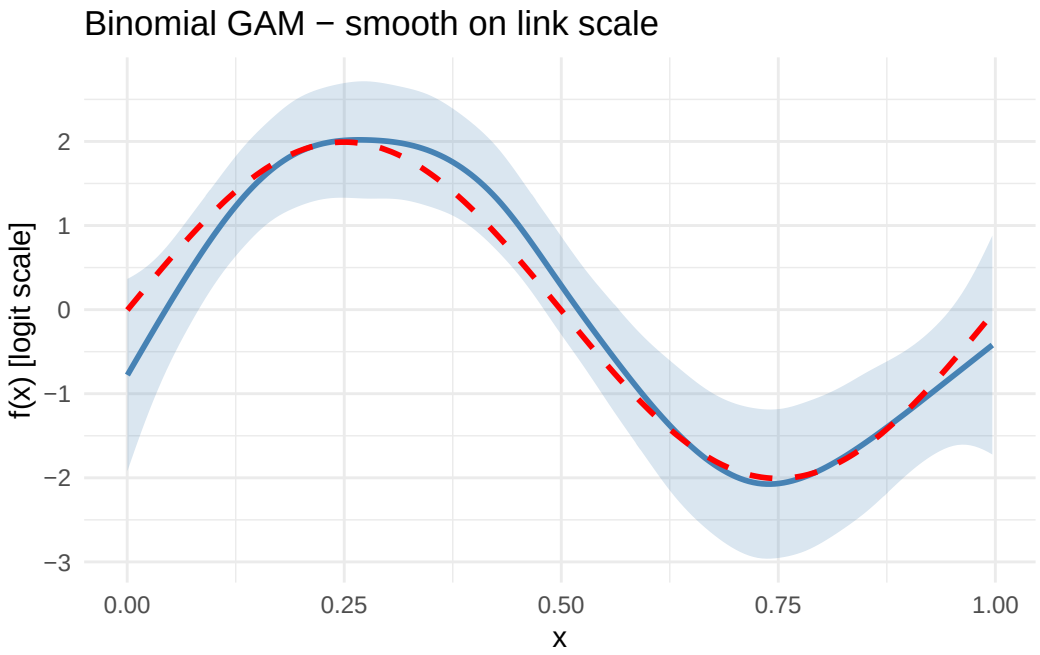
Fitting the model

```
m_bin <- gam(y ~ s(x, k = 15, bs = "cr"), data = df_bin,
             family = binomial(link = "logit"), method = "REML")
summary(m_bin)
```

```
Family: binomial
Link function: logit

Formula:
y ~ s(x, k = 15, bs = "cr")

Parametric coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -0.5190     0.1537  -3.378 0.000731 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
       edf Ref.df Chi.sq p-value
s(x) 5.204  6.432  82.61  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.347   Deviance explained =   29%
-REML =  152.5  Scale est. = 1         n = 300
```

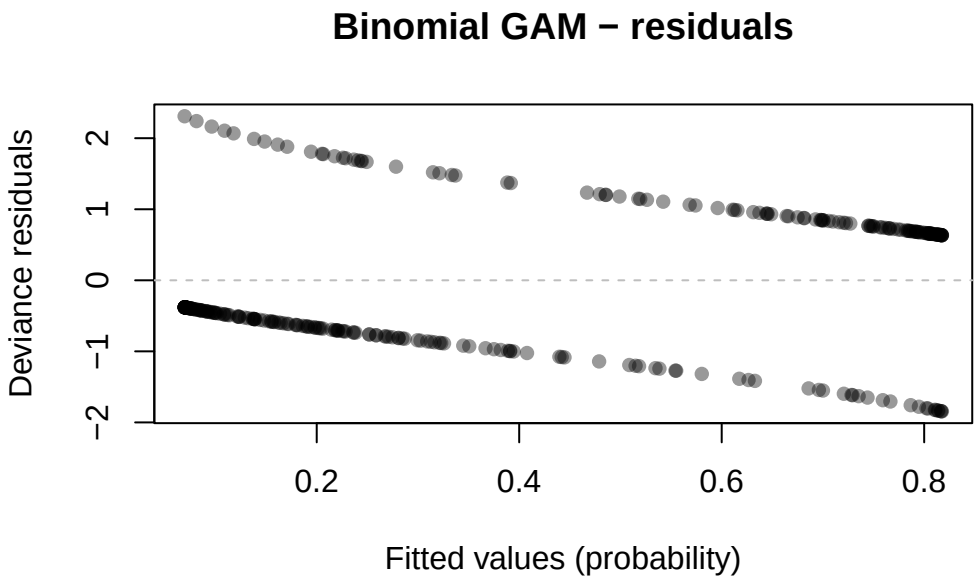
Smooth estimate

```r
se_bin <- smooth_estimates(m_bin, n = 200)

ggplot(se_bin, aes(x = x, y = .estimate)) +
  geom_ribbon(aes(ymin = .estimate - 2 * .se, ymax = .estimate + 2 * .se),
              alpha = 0.2, fill = "steelblue") +
  geom_line(linewidth = 1, colour = "steelblue") +
  geom_line(data = data.frame(x = x_bin, y = eta_bin - mean(eta_bin)),
            aes(x = x, y = y), linetype = "dashed", linewidth = 1, colour = "red") +
  labs(x = "x", y = "f(x) [logit scale]", title = "Binomial GAM - smooth on link scale") +
  theme_minimal()
```



Deviance residuals

```
resid_bin <- residuals(m_bin, type = "deviance")

plot(fitted(m_bin), resid_bin,
     xlab = "Fitted values (probability)", ylab = "Deviance residuals",
     main = "Binomial GAM - residuals", pch = 16, col = adjustcolor("black", 0.4))
abline(h = 0, lty = 2, col = "grey")
```



Gamma GAM — positive continuous data

Simulating positive continuous data

We simulate positive continuous data from a Gamma distribution with a smooth log-mean:

$$y_i \sim \text{Gamma}(\text{shape}, \text{scale}_i), \quad \log(\mu_i) = 1 + \sin(2\pi x_i)$$

```
set.seed(42)
x_gam <- sort(runif(n))
eta_gam <- 1.0 + sin(2 * pi * x_gam)
mu_gam <- exp(eta_gam)
shape <- 5.0
y_gam <- rgamma(n, shape = shape, scale = mu_gam / shape)

df_gam <- data.frame(x = x_gam, y = y_gam)
```

Fitting the model

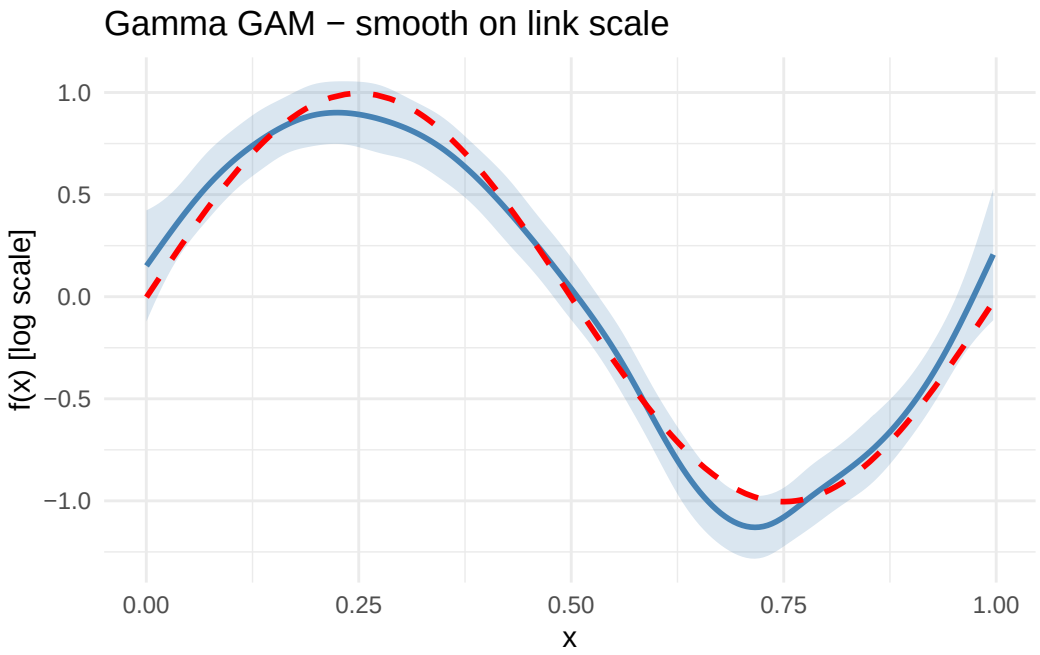
We use `Gamma` with `log` link (more commonly used in practice than the canonical inverse link):

```r
m_gam <- gam(y ~ s(x, k = 15, bs = "cr"), data = df_gam,
             family = Gamma(link = "log"), method = "REML")
summary(m_gam)
```

```
Family: Gamma
Link function: log

Formula:
y ~ s(x, k = 15, bs = "cr")

Parametric coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.95767    0.02741   34.94   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
       edf Ref.df     F p-value
s(x) 7.659  9.307 70.93  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =   0.56   Deviance explained = 66.8%
-REML = 477.56  Scale est. = 0.22545   n = 300
```

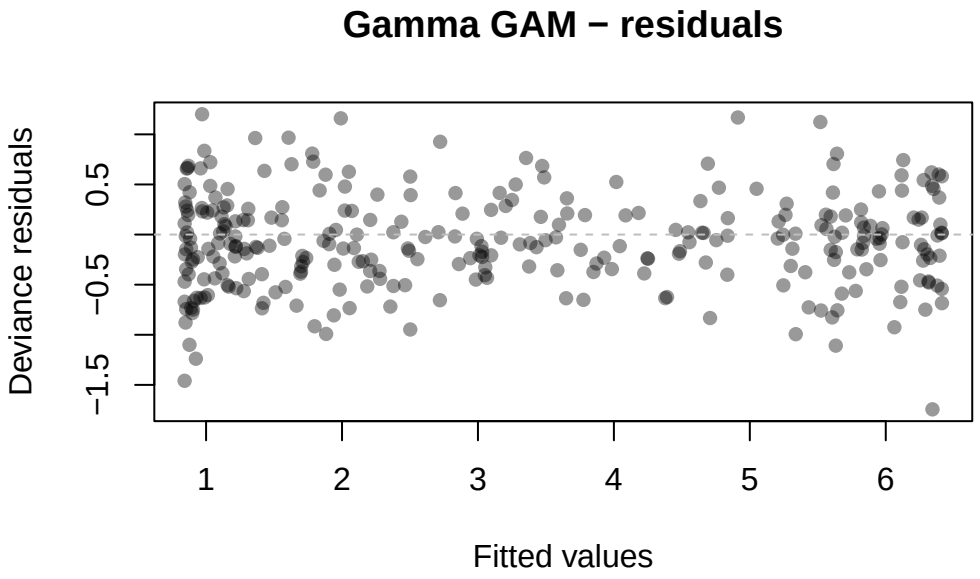
Smooth estimate

```r
se_gam <- smooth_estimates(m_gam, n = 200)

ggplot(se_gam, aes(x = x, y = .estimate)) +
  geom_ribbon(aes(ymin = .estimate - 2 * .se, ymax = .estimate + 2 * .se),
              alpha = 0.2, fill = "steelblue") +
  geom_line(linewidth = 1, colour = "steelblue") +
  geom_line(data = data.frame(x = x_gam, y = eta_gam - mean(eta_gam)),
            aes(x = x, y = y), linetype = "dashed", linewidth = 1, colour = "red") +
  labs(x = "x", y = "f(x) [log scale]", title = "Gamma GAM - smooth on link scale") +
  theme_minimal()
```



Deviance residuals

```
resid_gam <- residuals(m_gam, type = "deviance")

plot(fitted(m_gam), resid_gam,
     xlab = "Fitted values", ylab = "Deviance residuals",
     main = "Gamma GAM - residuals", pch = 16, col = adjustcolor("black", 0.4))
abline(h = 0, lty = 2, col = "grey")
```



Comparison

```
results <- data.frame(
  Family = c("Poisson", "Binomial", "Gamma"),
  EDF = c(
    round(sum(pen.edf(m_pois)), 2),
    round(sum(pen.edf(m_bin)), 2),
    round(sum(pen.edf(m_gam)), 2)
  ),
  Dev_Explained = c(
    round(summary(m_pois)$dev.expl * 100, 1),
    round(summary(m_bin)$dev.expl * 100, 1),
    round(summary(m_gam)$dev.expl * 100, 1)
  ),
  Scale = c(
    round(summary(m_pois)$scale, 4),
    round(summary(m_bin)$scale, 4),
    round(summary(m_gam)$scale, 4)
  )
)
print(results)
```

```
    Family  EDF Dev_Explained  Scale
1  Poisson 8.00          76.4 1.0000
```

```
2 Binomial 5.20          29.0 1.0000
3    Gamma 7.66          66.8 0.2254
```

Summary

In this vignette we:

1. Simulated count data and fitted a **Poisson GAM** with a log link
2. Simulated binary data and fitted a **Binomial GAM** with a logit link
3. Simulated positive continuous data and fitted a **Gamma GAM** with a log link
4. Examined smooth estimates and deviance residuals for each family
5. Compared EDF and deviance explained across families

Each family uses a different link function to map the linear predictor to the mean of the response distribution, but the smooth estimation machinery is the same.